

Apuntes de
Diseño y Análisis de Algoritmos
Tomados como estudiante y revisados como
profesor

Federico Melo Barrero
f.melo@uniandes.edu.co

Copyright © 2022, 2026 Federico Melo Barrero.

Todos los derechos reservados.

Ninguna parte de este documento, incluyendo el diseño de la portada, figuras e imágenes, puede ser reproducida, registrada, almacenada o transmitida en manera alguna ni por ningún medio, ya sea electrónico, eléctrico, fotoquímico, mecánico, electroóptico, magnético, de grabación, de fotocopia o cualquier otro sin permiso previo y por escrito del autor.

Índice

1. Prerrequisitos	3
2. Definiciones básicas	3
2.1. Algoritmo	3
2.2. Seudocódigo	4
2.3. Problema	4
 I Diseño de algoritmos	 5
3. Dividir y conquistar	5
3.1. Merge sort	5
3.2. Subarreglo máximo	6
3.2.1. Solución por fuerza bruta	6
3.2.2. Solución por dividir y conquistar	7

Preámbulo

Tomé estos apuntes cuando cursé Diseño y Análisis de Algoritmos en la Universidad de los Andes, entre agosto y noviembre de 2022.

Posteriormente los fui corrigiendo y extendiendo de intermitentemente a medida que trabajaba en temas relacionados con algoritmia. Las extensiones más notables corresponden a dos momentos: cuando cursé Teoría de Grafos entre agosto y noviembre de 2023, y cuando fui seleccionado para dictar el curso de Diseño de Algoritmos en mayo de 2026.

Los apuntes están presentados en el mismo formato en el que originalmente fueron escritos: la clase de $\text{\LaTeX}$ que establece su estilo la escribí en noviembre del 2020. Más importante aún, los apuntes preservan la estructura y el tono de cuando los escribí como estudiante, con la intención de que sean útiles precisamente a esa audiencia.

La selección deliberada de la audiencia es en gran medida lo que otorga valor a estas notas. Textos clásicos como CLRS¹ intentan dirigirse a una audiencia mixta: estudiantes, profesores, investigadores y profesionales. Esa amplitud de audiencia genera que el texto se difunda ampliamente, pero también implica un nivel de formalidad, profundidad y exhaustividad que muchas veces no es el más adecuado para un primer encuentro con el tema, especialmente a nivel de pregrado.

Estas notas no sacrifican rigor ni correctitud, pero sí priorizan claridad pedagógica sobre exhaustividad formal. En consecuencia, omiten algunos detalles cuya inclusión sería valiosa en un texto de referencia, pero que para muchos estudiantes solo alargarían el texto y añadirían complejidad innecesaria.

1. Prerrequisitos

Estos apuntes están escritos de forma que los prerrequisitos para entenderlos son sorprendentemente laxos para el nivel de material que se cubre. Hay que saber lo siguiente:

- Programación básica en algún lenguaje de programación, incluyendo recursión.
 - El pseudocódigo que se emplea es bastante parecido a Python.
- Algo de teoría sobre las estructuras de datos más básicas: arreglos, listas, etc. En qué difieren y cuáles son las ventajas de una sobre otra.
- Notación matemática básica (conjuntos, operadores, etc.), aritmética básica (sumar, multiplicar, etc.), álgebra elemental (despejar variables, factorizar, etc.) y funciones elementales (logaritmos, exponenciales, etc.).

2. Definiciones básicas

2.1. Algoritmo

Empezamos suave, con algunas definiciones básicas.

Un **algoritmo** es una secuencia de pasos. Pero nos interesan los algoritmos que resuelven un problema de computación, entonces:

Definición 2.1. Algoritmo

Secuencia de pasos para transformar un conjunto de valores (**entrada/input**) a un conjunto de valores (**salida/output**).

¹Uso esta sigla para referirme al famosísimo Introduction to Algorithms de Cormen, Leiserson, Rivest y Stein, que es un excelente ejemplo de este fenómeno: con docenas de miles de citas en google scholar, claramente le resulta útil a investigadores.

No se escribe “a otro conjunto de valores” porque la salida podría ser igual a la entrada.

Un algoritmo se puede expresar en:

- Lenguaje natural (español, inglés, etc.)
- Lenguaje formal (matemática o algún lenguaje de programación o lógico)
- Seudocódigo (una mezcla de los dos anteriores)
- Como programa de computador
- Como hardware (esto es difícil de imaginar en este punto, pero sí)

2.2. Seudocódigo

El pseudocódigo es básicamente algo que se parece a un lenguaje de programación pero que permite:

- Despreocuparse de detalles de sintaxis o de implementación, como el manejo de errores.
- Usar la forma más clara de expresarse en cada línea: a veces español, a veces matemática, a veces sintaxis de algún lenguaje de programación.

Entonces es muy útil para expresar algoritmos.

En los textos formales se definen convenciones estrictas para el pseudocódigo que se usa; eso elimina la ambigüedad, pero obliga al lector a aprenderse las convenciones, prácticamente un nuevo lenguaje. Contrariamente, en estos apuntes se utiliza un pseudocódigo menos estructurado, basado en la sintaxis de Python, pero con varias libertades y las ambigüedades las elimino aclarándolas cuando aparecen.

E.g., una función que recibe un arreglo de números y encuentra el mayor:

```

1 función encontrar_mayor(números: arreglo)
2     mayor = -∞
3     for i=0 to números.longitud-1
4         if números[i] > mayor
5             mayor = números[i]
6     return mayor

```

Detalles de este pseudocódigo que reflejan que su propósito es ser claro:

- Algunas palabras y tipos están en español. A pesar de que no se definieron de antemano, son muy claros porque son los términos que usamos para referirnos a esas cosas.
- Omite detalles de sintaxis como los dos puntos, pero preservo los que otorgan claridad, como la indentación.
- Evito sintaxis específica de algún lenguaje.
 - En el `for` uso una sintaxis que hace explícito el valor inicial y el valor final sin necesidad de usar iterables (y sin tener que desambiguar si se toma o no el último valor; acá es claro que sí)
 - Presumo la existencia de un método `longitud` para obtener la longitud de un arreglo. Es una presunción razonable porque en cualquier lenguaje orientado a objetos existe algo así.

2.3. Problema

Un problema (en este contexto) es un enunciado que especifica qué se quiere obtener, sin decir cómo. Más formalmente:

Definición 2.2. Problema (computacional)

Especificación de una entrada y de una salida con base en la entrada.

E.g., el **problema de ordenamiento** (*sorting problem*, muy famoso):

- **Entrada:** secuencia de n números $a_1, a_2, \dots, a_n$.
- **Salida:** permutación $a'_1, a'_2, \dots, a'_n$ tal que $a'_1 < a'_2 < \dots < a'_n$

Una permutación de una secuencia es un posible orden de sus elementos. Si la secuencia tiene más de un elemento, tiene muchas permutaciones.

Una **instancia** de un problema es una entrada específica, e.g., 1, 4, 2, 3 es una instancia del problema de ordenamiento.

Parte I

Diseño de algoritmos

3. Dividir y conquistar

Muchos problemas pueden dividirse en subproblemas más pequeños y esos subproblemas siguen siendo muy similares al problema original. Por ejemplo, organizar su casa se puede dividir en organizar la cocina, la sala, etc. que son problemas parecidos pero con menos cosas para organizar.

Dividir y conquistar es la técnica de diseño de algoritmos que formaliza esa idea. Consiste en dividir el problema en subproblemas, resolver cada uno de ellos y luego combinar las soluciones para obtener la solución del problema original. Formalmente, las etapas son:

- **Dividir:** Dividir el problema en subproblemas más pequeños
- **Conquistar:** Resolver cada subproblema de forma recursiva (o, si el subproblema es muy pequeño, resolverlo directamente)
- **Combinar:** Combinar las soluciones de los subproblemas, obteniendo la solución del problema original

Se presentan tres algoritmos para concretar esta idea.

3.1. Merge sort

Merge sort resuelve el problema de ordenamiento (definido en la sección 2.3).

La idea es dividir el arreglo a ordenar en dos mitades, ordenar cada mitad de forma recursiva (es decir, dividiendo cada una otra vez a la mitad, y así sucesivamente) y luego combinar las soluciones.

Haciendo explícitas las etapas:

- **Dividir:** Dividir el arreglo en dos mitades.
- **Conquistar:** Ordenar cada mitad usando merge sort (o sea, aplicando estas etapas a cada mitad)
- **Combinar:** Combinar las soluciones de cada mitad, que son subarreglos ordenados.

Los primeros dos pasos son muy sencillos porque en algún punto al dividir por la mitad se obtienen arreglos de un elemento, que ya no hay que ordenar. Entonces, si encapsulamos el paso de combinar, este es el pseudocódigo:

```

1 función merge_sort(números: arreglo, pos_min: entero, pos_max: entero)
2     if pos_min >= pos_max
3         return números
4
5     pos_mitad = (pos_min + pos_max) // 2
6     merge_sort(números, pos_min, pos_mitad)
7     merge_sort(números, pos_mitad+1, pos_max)
8
9     combinar(números, pos_min, pos_mitad, pos_max)

```

Necesitamos tener los índices como parámetros porque indican cómo se va dividiendo el arreglo. La invocación principal, que resuelve el problema, requiere los índices extremos del arreglo: `merge_sort(números, 0, números.longitud-1)`. Nótese que en esta función el arreglo realmente no se divide: las llamadas recursivas trabajan sobre el mismo arreglo con distintos índices, la división es conceptual.

Sorprendentemente, la mejor forma de combinar las soluciones es la forma más intuitiva. Si cada arreglo fuera una pila de cartas, el método natural sería comparar la primera de cada pila y tomar la menor; luego

comparar la siguiente de ese arreglo con la primera del otro arreglo y tomar la menor (que es la que va después de la primera que tomé), y así sucesivamente. El pseudocódigo es el siguiente:

```

1 función combinar(números: arreglo, pos_min: entero, pos_mitad: entero, pos_max: entero)
2   izquierda = copia de números desde pos_min hasta pos_mitad
3   derecha = copia de números desde pos_mitad+1 hasta pos_max
4   i = 0; j = 0
5
6   # Ambos subarreglos tienen elementos
7   while i < izquierda.longitud and j < derecha.longitud
8     if izquierda[i] <= derecha[j] # <= y no < para estabilidad
9       números[pos_min+i+j] = izquierda[i]
10      i++
11     else
12       números[pos_min+i+j] = derecha[j]
13       j++
14
15   # Elementos restantes de la izquierda
16   while i < izquierda.longitud
17     números[pos_min+i+j] = izquierda[i]
18     i++
19
20   # Elementos restantes de la derecha
21   while j < derecha.longitud
22     números[pos_min+i+j] = derecha[j]
23     j++

```

Un algoritmo es **estable** si mantiene el orden relativo de los elementos iguales. O sea, en caso de empate, deja primero al que estaba primero en el arreglo original. En la línea del comentario sobre estabilidad, se elige el elemento de la izquierda en caso de empate para que continúe ubicado antes que el de la derecha en la solución final.

3.2. Subarreglo máximo

Dado un arreglo de números, a un subarreglo contiguo no vacío cuya suma (de sus elementos) es mayor que la de cualquier otro subarreglo contiguo se denomina un **subarreglo máximo**. El problema de encontrar un subarreglo máximo se define como:

- **Entrada:** arreglo con n números: $[a_1, a_2, \dots, a_n]$.
- **Salida:** subarreglo contiguo $[a_i, a_{i+1}, \dots, a_j]$ no vacío tal que la suma $\sum_{k=i}^j a_k$ es máxima.

“es máxima” se puede formalizar en una expresión matemática que básicamente dice que esa sumatoria es mayor que cualquier otra sumatoria de elementos contiguos que se pueda formular. Nótese que podría haber más de un subarreglo máximo: podrían haber varios cuya suma sea igual. Basta con hallar cualquiera que sea máximo.

Hay dos tipos de instancias de este problema que son muy fáciles:

- Cuando todos los valores del arreglo son positivos, la respuesta es todo el arreglo, porque todos los valores aportan a una mayor suma.
- Cuando todos los valores son negativos, se quiere que la suma sea lo menos negativa, por lo que se forma el subarreglo con un solo elemento, el menos negativo.

3.2.1. Solución por fuerza bruta

Una primera solución a este problema se consigue simplemente comparando todas las posibilidades de subarreglos y eligiendo el que tiene la mayor suma:

```

1 funcion subarreglo_max_fuerza_bruta(numeros: arreglo)
2     mayor = -∞
3     pos_ini = 0
4     pos_fin = 0
5
6     for i=0 to numeros.longitud-1
7         suma = 0
8         for j=i to numeros.longitud-1
9             suma += numeros[j]
10
11             if suma > mayor
12                 mayor = suma
13                 pos_ini = i
14                 pos_fin = j
15
16     return pos_ini, pos_fin, mayor

```

Ese algoritmo cuenta con dos ciclos anidados que recorren el arreglo, por lo que intuitivamente su complejidad temporal es $O(n^2)$.

3.2.2. Solución por dividir y conquistar

Para diseñar un algoritmo usando dividir y conquistar, el primer paso convencional es partir el arreglo. Con eso, cada subproblema es hallar un subarreglo máximo en el respectivo pedazo. El cuidado que hay que tener con esa solución es que, al partir el arreglo, podríamos tener el infortunio de partirlo justo donde estaba el subarreglo máximo, entonces en el paso de combinar las soluciones, tenemos que asegurarnos de contemplar esa posibilidad. Con eso, las etapas son:

- **Dividir:** Dividir el arreglo en dos mitades.
- **Conquistar:** Para cada mitad, hallar un subarreglo máximo recursivamente.
- **Combinar:** Comparar los subarreglos máximos de cada mitad para elegir el mayor y también compararlos con los subarreglos que pasan por donde se dividió el arreglo.

Si encapsulamos la última parte del paso de combinar en `subarr_max_transversal`, de forma que aplazamos un poco ese problema, podemos escribir el pseudocódigo para las primeras dos etapas:

```

1 funcion subarr_max_div_conquistar(numeros: arreglo, pos_min: entero, pos_max: entero)
2     if pos_min == pos_max
3         return pos_min, pos_max, numeros[pos_min]
4
5     pos_mitad = (pos_min + pos_max) // 2
6
7     ini_izq, fin_izq, suma_izq = subarr_max_div_conquistar(numeros, pos_min, pos_mitad)
8     ini_der, fin_der, suma_der = subarr_max_div_conquistar(numeros, pos_mitad+1, pos_max)
9     ini_mitad, fin_mitad, suma_mitad = subarr_max_transversal(numeros, pos_min, pos_mitad, pos_max)
10
11     if suma_izq >= suma_der and suma_izq >= suma_mitad
12         return ini_izq, fin_izq, suma_izq
13     if suma_der > suma_mitad
14         return ini_der, fin_der, suma_der
15     return ini_mitad, fin_mitad, suma_mitad

```

Ahora sí, pensemos los subarreglos que pasan por donde se dividió el arreglo. Todos esos subarreglos tienen un pedazo a la izquierda de esa mitad y un pedazo a la derecha. Para maximizar la suma, queremos maximizar la de la derecha y maximizar la de la izquierda. En este caso, usar un algoritmo exhaustivo no es ingenuo:


```
1 funcion subarr_max_transversal(  
2     numeros: arreglo,  
3     pos_min: entero,  
4     pos_mitad: entero,  
5     pos_max: entero  
6 )  
7     suma_der, suma_izq = 0, 0  
8     mayor_der, mayor_izq =  $-\infty$ ,  $-\infty$   
9  
10    for i=pos_mitad to pos_max  
11        suma_der += numeros[i]  
12        if suma_der > mayor_der:  
13            mayor_der = suma_der  
14            extremo_der = i  
15  
16    for j=pos_mitad-1 downto pos_min  
17        suma_izq += numeros[j]  
18        if suma_izq > mayor_izq:  
19            mayor_izq = suma_izq  
20            extremo_izq = j  
21  
22    return extremo_izq, extremo_der, mayor_izq+mayor_der
```